

ForgeCloud Product & Requirements

Product definition, modules, user journey and measurable outcomes

1. Product Overview

ForgeCloud is a self-service internal developer platform that allows developers to create, deploy, monitor, scale and manage applications through a web-based control plane. It automates infrastructure provisioning and application deployment using Infrastructure as Code, containers, Kubernetes, CI/CD and GitOps.

2. Product Goals

- Self-service deployment through a web dashboard.
- Infrastructure provisioning with Terraform rather than manual AWS console configuration.
- Containerized workloads using Docker and Kubernetes/Amazon EKS.
- Automated testing, image building and deployment using GitHub Actions.
- GitOps synchronization using Argo CD.
- Unified observability using Prometheus, Grafana, OpenTelemetry and CloudWatch.
- Reliability through health checks, autoscaling, rollback and controlled failure testing.
- Security through authentication, RBAC, IAM, secrets management and network controls.

3. Core User Journey

Developer

- > ForgeCloud Dashboard
- > Create Application
- > Configure runtime/resources
- > Trigger deployment
- > GitHub Actions
- > Docker image
- > Amazon ECR
- > Argo CD
- > Kubernetes/EKS
- > Application running
- > Metrics/logs/traces
- > ForgeCloud monitoring dashboard

4. Product Modules

Module	Main responsibility
Authentication & RBAC	Login, authorization and role separation
Application Management	Create, configure and manage applications
Infrastructure Provisioning	Terraform-based AWS resource provisioning
Container Build & Registry	Docker packaging and Amazon ECR
Deployment Engine	Kubernetes manifests/Helm and Argo CD
Networking	VPC, subnets, routes, security groups, load balancing, ingress
Autoscaling	Kubernetes HPA based on resource utilization

Module	Main responsibility
Observability	Metrics, logs and traces
Rollback & Reliability	Health checks, failed-release detection and restoration

5. Signature Features

- Self-service deployment without manually creating every cloud resource.
- Automatic scaling when workload/resource utilization increases.
- Deployment rollback to a previous known-good version.
- Unified visibility of metrics, logs and traces.
- Network-aware request flow through AWS and Kubernetes layers.
- Reproducible infrastructure through Terraform.

6. Demonstration Scenario

A developer creates an application in ForgeCloud, supplies its repository and runtime configuration, and triggers deployment. The platform builds the container, publishes it to ECR, synchronizes Kubernetes configuration through Argo CD, exposes the workload through controlled networking, and presents deployment and observability information in the dashboard.

7. Evidence to Collect

- Deployment time before and after automation.
- API latency and application P95 latency.
- Throughput under load.
- Autoscaling response and replica count.
- Resource utilization.
- Rollback and recovery time.
- Error rate and availability.
- Infrastructure provisioning success rate.